

Q&A

Object Lessons

The creator of the Eiffel programming language discusses his career in industry and academia, “Design by Contract,” and his views on Agile software development.

FRENCH-BORN COMPUTER SCIENTIST Bertrand Meyer—best known as an early advocate of object-oriented programming techniques and creator of the programming language and environment Eiffel—has enjoyed a varied career in industry and academia. Currently a professor of software engineering at ETH Zurich, the Swiss Federal Institute of Technology, he also has worked at Électricité de France (EDF) and at the University of California, Santa Barbara, and he continues to serve as CEO and chief architect at the California-based company he founded in 1985, Eiffel Software.

After receiving degrees from Stanford and the University of Nancy, you spent nearly 10 years in industry at Électricité de France (EDF). When did you begin working on Eiffel, the object-oriented language and environment that you continue to refine and develop?

In 1983, I got to spend a sabbatical at the University of California, Santa Barbara. A Japanese company got excited about a structured editor we had developed, and in 1985, we decided to found a company that’s now called Eiffel Software.

We were looking for a programming language. We didn’t like what was available, so I designed a notation that became Eiffel. Initially, I didn’t pay much attention to it. But in 1986, we attended the first OOPSLA (Object-Oriented Programming, Systems, Languages, and Applications) conference in Portland, and that’s where we realized that what I thought obvious was



Bertrand Meyer, professor of software engineering at ETH Zurich, and CEO and chief architect of California-based Eiffel Software.

actually ahead of the curve. So coming back from Portland, we refocused the company on Eiffel.

You are also known for the idea of “Design by Contract,” a method of assuring a program’s correctness by specifying the conditions that each element must satisfy; there are preconditions, which state what an operation expects, and postconditions, which state what an operation guarantees. Did that idea evolve in tandem with Eiffel?

Yes. After reading the works of Hoare and Dijkstra, it seemed like a direct, practical application of everything

we knew about program correctness. Design by Contract is a transposition to software of concepts that everyone is familiar with. If I want to buy something from you, I have a certain set of obligations to satisfy, and on your side, you also have obligations. Your obligations map into my benefits and conversely. So, too, with preconditions and postconditions in software.

Let’s talk about your move to ETH Zurich, where you’ve been since 2001.

It was not planned, but it has worked out very nicely. In 2000, Niklaus [CONTINUED ON P. 95]

[CONTINUED FROM P. 96] Wirth had retired from ETH and I received an invitation to take on a chair. It took me a year to organize the transfer, because I had my company to deal with, and I couldn't just drop it overnight. And it's a long flight from Santa Barbara to Zurich. Nonetheless, I dived headfirst into the academic world.

Yet you have remained involved with Eiffel Software.

The trick for me has been not to develop a dual personality. I'm the same person whether I work as a professor or whether I'm devoting time to the company, and my research is still in the context and philosophy of Eiffel. If you want to survive as a company, you have to do what the customers want. In academia, you can play with crazy ideas and experiment without regard to what you can sell. You're not going to hit the scientific jackpot every time. But once in a while, you are ahead of the game and do things that you cannot do in a company if you are focused on the bottom line.

What specific elements of EiffelStudio have evolved through your academic work?

Eiffel offers you the ability to test programs automatically—and it really is completely automatic in the sense that everything is generated by the software, even the test cases. That started out in an academic context at ETH and was refined through several excellent Ph.D. theses.

The much more ambitious goal, which has been made possible by a succession of Ph.D. theses, is the idea of fully verified software. For a long time, that looked like a purely intellectual pursuit, but it's now becoming a reality. We call it EVE, the Eiffel Verification Environment. There's still a lot of work to be done, and we have benefited from tremendous advances in technology, in particular the Boogie tools from Microsoft Research. But it's becoming realistic to imagine that we can take large, real-life programs and prove that they're correct. And if they're not correct, we can also—that's another part of the EVE research—automatically suggest fixes.

On the teaching side, you helped produce a MOOC (Massive Open Online Course) that enables students any-

“The trick for me has been not to develop a dual personality. I'm the same person whether I work as a professor or whether I'm devoting time to the company.”

where to take the introductory programming course offered at ETH.

Coming from industry, the last thing I expected to do was to teach bright-eyed 19-year-olds the rudiments of programming, let alone in German, or my imitation of it. Yet it has been one of the most exciting things I have done at ETH. I started looking in depth at how we can teach programming today to kids who have been using smartphones and videogames all their lives, and need the competitive advantage of becoming true professionals.

I use Eiffel and Design by Contract right from the start. The experience resulted in a textbook, *Touch of Class*, and more recently I dived head-on into MOOCs, first producing a kind of skunkworks MOOC outside of any organizational structure at the initiative of my colleague Marco Piccioni. Now we have redone it in a completely official context and it's an edX offering. It applies the same pedagogical principles as our course and also benefits from our research on distributed software development; students can compile and run programming exercises online, as quizzes in the course, and see the results right away.

You also recently published a book on Agile development methods with the subtitle “The Good, the Hype and the Ugly.”

Usually, when you see a novel idea in software engineering, you can quickly recognize whether it's good or bad. What's special about Agile is that it's a mix of the best and the worst. My book is a cool-headed attempt to separate the wheat from the chaff.

What are the best elements of Agile, in your opinion?

Among the best is the idea of developing in short iterations of two to six weeks. This has profoundly transformed the software industry for the better, and no one develops software anymore by assembling a few groups who tackle different parts of the program and will see each other in six months.

Another example is what I call the closed-window rule, an absolutely brilliant idea—that when you have an iteration, you schedule a certain number of tasks, and absolutely no one, regardless of rank, is permitted to add anything during that iteration. This rule has a number of benefits. It stabilizes the whole process and prevents bosses and managers from interrupting the iteration. It also has the benefit of weeding out bad ideas, because many suggestions that seem brilliant don't look so good when you wake up sober the next day.

And the worst?

In the opposite camp, you have the general rejection of what's derisively called “big upfront anything”—big upfront requirements, big upfront design. The Agile credo is that you should start implementing part of the system right away and not engage in long, early phases of architecture or investigation. The Agile world has this phobia of not producing anything that's not deliverable to the customer. And as is so often the case with Agile ideas, there's a grain of truth in this rejection, because the customer does not need specifications. The customer needs results. But the idea that it's bad to spend an appropriate time at the beginning of the project to clarify the overall requirements and design is nonsense. You do need at some point to focus on the deliverables; but until then you should take all the time necessary to address the big issues of specification and design. I've seen projects fail miserably for blindly applying the Agile catechism: we're Agile, we don't need to stop and think, we just go ahead and code! Not surprisingly, what they code is junk and they have to redo it, but maybe at that point the money has run out and the customer has lost faith. ■

Leah Hoffmann is a technology writer based in Piermont, NY.

© 2015 ACM 0001-0782/15/03 \$15.00